

软件工程基础第 7 章—详细设计

学习笔记（精炼版）

中南大学计算机学院 任胜兵

2026 年 6 月 1 日

目录

1	详细设计的目标与任务	2
2	详细设计图形描述工具	2
2.1	程序流程图（Program Flowchart）	2
2.2	盒图 / N-S 图（Box Diagram）	3
2.3	问题分析图（PAD / Problem Analysis Diagram）	3
2.4	协作图（Collaboration Diagram）	4
3	Jackson 结构化程序设计方法（JSP）	5
3.1	Jackson 图（Jackson Diagram）	5
3.2	Jackson 伪代码	5
3.3	JSP 设计五步骤	5
3.4	完整示例——装修公司构件周报表	6
4	Warnier 程序设计方法（LCP）	6
4.1	Warnier 图（Warnier Diagram）	6
4.2	LCP 设计四步骤	6
5	面向对象程序的详细设计	7
5.1	三大特性	7
5.2	四大设计原则	8
6	程序规格说明文档及复审	8
7	小结	9

1 详细设计的目标与任务

[p.1-4]

定位

详细设计（Detailed Design）是编码前最后一个设计阶段，将概要设计的模块划分细化为可直接编码的精确描述。结构化设计和面向对象设计在此阶段**本质相同**——都是确定算法和数据结构。产出物为**软件详细规格说明**（Software Detailed Specification），相当于施工图纸。[p.1-2]

目标与任务

[p.3-4]

目标：得出精确、简明易懂的软件详细规格说明，高质量完成详细设计是提高软件质量的关键。

六项任务：

1. 算法过程设计（Algorithm Design）—模块内部的执行逻辑与控制流程
2. 数据结构设计（Data Structure Design）—数组、链表、树等数据组织方式
3. 数据库物理设计（Physical DB Design）—逻辑模型到物理存储的映射
4. 信息编码设计（Coding Design）—错误码、状态码等编码规则
5. 测试用例设计（Test Case Design）—基于详细设计提前设计白盒测试用例
6. 编写详细设计说明书（Detailed Design Document）

2 详细设计图形描述工具

[p.5-13]

四种核心图形工具用于过程设计，也可辅以 IPO 图、判定表/判定树、伪代码等。[p.5]

2.1 程序流程图（Program Flowchart）

[p.6-7]

符号体系

- 矩形框：处理步骤；菱形框：条件判断
- 箭头连线：控制流方向；圆角矩形：起止
- 平行四边形：输入/输出

优缺点

- 优点：直观灵活、易学易用、语言无关
- 缺点：不利于逐步求精，箭头可任意转移导致非结构化设计（GOTO 式），不便于表示数据结构

适用场景：小型简单模块的算法描述。

2.2 盒图 / N-S 图 (Box Diagram)

[p.8-9]

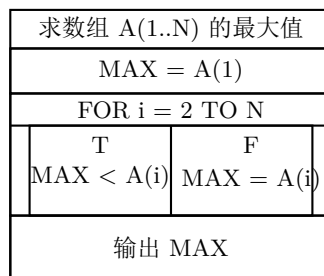
核心理念

由 Nassi 和 Shneiderman (1973) 提出。用盒状嵌套代替箭头控制流，将控制结构的作用域严格限定在盒子内，从机制上杜绝非结构化转移。

四种基本控制结构

1. 顺序：框从上到下垂直排列
2. 选择 (if-then-else)：条件标记分隔框
3. 循环 (while-do / repeat-until)：循环条件标记框
4. 调用：子程序调用标记框

示例——求数组最大值



优缺点

- 优点：强制结构化、控制作用域清晰、可附加数据盒子
- 缺点：修改困难、嵌套深时不易绘制、普及度不高

2.3 问题分析图 (PAD / Problem Analysis Diagram)

[p.10-11]

核心理念

由日本日立公司二村良彦 (1979) 提出。采用树形结构表示程序逻辑，纵向为主干（顺序执行），横向展开分支（选择和循环）。

基本结构

- 顺序：处理框沿纵向依次排列
- 选择：条件框横向引出分支线，标注条件值
- 循环：纵向延伸框标注 while/until 条件
- def：定义子过程调用

核心优势

1. **强制结构化：**与盒图相同，不会产生非结构化控制流
2. **树形层次清晰：**克服了流程图不能表现层次结构的缺点
3. **支持自动代码生成：**对 PAD 图进行**树遍历**即可自动生成程序代码
4. **便于修改：**树形结构使局部修改不影响整体

三种工具对比

维度	程序流程图	盒图/N-S 图	PAD 图
结构化保证	无（支持任意转移）	强制	强制
层次表达	弱	盒子嵌套	树形清晰
修改便利性	灵活	困难	较方便
代码生成	不支持	不支持	支持（树遍历）

2.4 协作图（Collaboration Diagram）

[p.12-13]

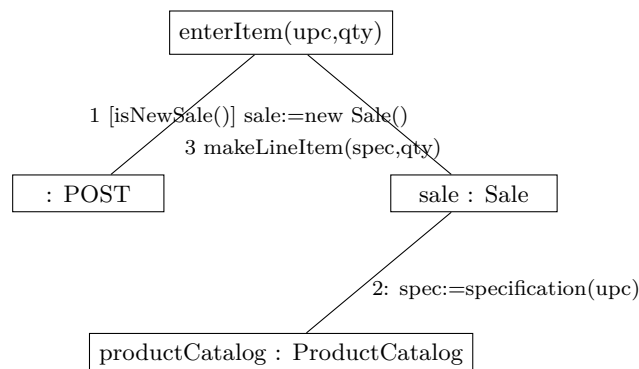
定义

UML 交互图之一，表示对象之间的**静态连接关系**和**消息传递**。与顺序图（Sequence Diagram）等价可互转。

构成要素

- **对象框：**objectName : ClassName 格式
- **连接线（Link）：**对象间的静态关联
- **消息：**沿连接线标注，格式为 序号方法名（参数）
- **序号（Sequence Number）：**嵌套编号（1, 1.1, 1.2, 2）表示消息层次

示例——商品录入系统



协作图 vs 顺序图

维度	协作图	顺序图
关注点	对象静态连接关系	消息传递时间顺序
时间表达	序号（无时间轴）	垂直生命线（时间感强）
空间效率	更省空间	对象多时水平展开

3 Jackson 结构化程序设计方法（JSP）

[p.14-23]

概述

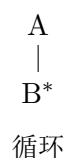
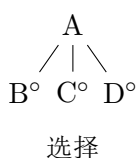
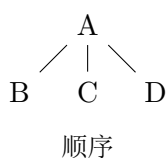
[p.14]

JSP（Jackson Structured Programming）由 M.A.Jackson（英国）提出，是面向数据结构的程序设计方法。核心理念：**数据结构决定程序结构**。适用于数据处理类系统（输入输出数据结构清晰）。

3.1 Jackson 图（Jackson Diagram）

[p.15] 树形层次图，用三种基本结构表示数据/程序：

1. 顺序（Sequence）：B, C, D 依次排列在 A 下，连线无标记
2. 选择（Selection）：B°, C°, D° 表示三者选一，连线标记小圆圈
3. 循环（Iteration）：B* 表示 B 出现 0 或多次，连线标记星号



3.2 Jackson 伪代码

[p.16]

- 顺序：A seq B; C; D end A
- 选择：A select cond B or c1 C or c2 D end A
- 循环：A iter while cond B end A

3.3 JSP 设计五步骤

[p.17]

1. 分析输入输出数据结构：用 Jackson 图表示
2. 找出对应关系：输入输出中有直接因果关系的单元。**关键**：重复单元必须“次序和次数都相同”才有对应关系

3. 映射为程序结构：按层次对应规则：高层在上、低层在下，处理框层次与数据结构中较低层一方对齐
4. 列出并分配操作与条件：列出全部操作（输入/计算/输出等）和条件，分配到程序结构图
5. 写出 Jackson 伪代码

3.4 完整示例——装修公司构件周报表

[p.18-23] 问题：根据构件记录表（编号、名称、数量、操作类型）生成周报表（编号、变化量）。

数据结构分析：

- 输入结构：构件记录表 → 构件记录 *（编号 / 数量 / 操作类型 [出库 ° | 入库 °]）
- 输出结构：周报表 → 表头 + 表体 → 行 *（编号 / 变化量）
- 对应关系：引入”构件组记录”桥接——同一编号的多条记录聚合为一个报表行

条件定义：I(1): 文件未到尾；I(2): 编号 当前 ID（换组）；S(3): 操作类型是出库

操作（共 17 项）：含文件打开/关闭、索引、读编号/数量、change±=data、生成表头/表行字符串等

4 Warnier 程序设计方法（LCP）

[p.24-31]

概述

[p.24]

LCP（Logical Construction of Programs）由 J.D.Warnier（法国）提出，同样面向数据结构。与 JSP 的差异在于花括号层次图代替树形图，且中间需转为程序流程图。

4.1 Warnier 图（Warnier Diagram）

[p.25]

使用从左到右的花括号嵌套表示层次组成：

- {：层次分解，花括号后是子成分
- ⊕（异或）：选择关系（二选一或多选一）
- (n)：重复次数标注

4.2 LCP 设计四步骤

[p.26-31]

1. 分析输入输出数据结构：用 Warnier 图表示
2. 导出程序处理结构：从输入数据结构推导，用 Warnier 图表示
3. 转为程序流程图：将 Warnier 程序结构转为结构化流程图
4. 写出详细过程性描述：

- 给流程图每个处理框统一编号
- 列出每框所需指令，分五类：A（输入）、B（分支）、C（计算）、D（子程序调用）、E（输出）
- 按框序号重排，同序号按“输入 → 处理 → 输出”排列

JSP vs LCP 对比

维度	JSP (Jackson)	LCP (Warnier)
提出者	M.A.Jackson（英国）	J.D.Warnier（法国）
图形工具	Jackson 树形图	Warnier 花括号层次图
文本输出	Jackson 伪代码	分类指令排序表
中间表示	直接用 Jackson 图	需先转为程序流程图

5 面向对象程序的详细设计

[p.32–36]

概述

[p.32]

面向对象详细设计完成对象属性（Attribute）和方法（Method）的设计。与结构化详细设计相似但需考虑 OOP 特性。

5.1 三大特性

[p.33–35]

1. 封装性（Encapsulation）

[p.33] 将数据和操作数据的方法捆绑在类内，通过访问控制（public/protected/private）限制作用域。

- 接口与实现分离；降低耦合；提高安全性和可维护性

2. 继承性（Inheritance）

[p.34] 子类（Derived Class）继承父类（Base Class）的成员函数和数据成员。

- 代码复用；层次化建模（is-a 关系）；支持扩展
- 单继承（Java/C#）vs 多重继承（C++）
- C++ 语法：`class CDerived : public CBase {}`

3. 多态性 (Polymorphism)

[p.35] 不同类的对象通过相同接口呈现不同行为。

- 编译时多态：函数/运算符重载 (Overloading)
- 运行时多态：虚函数 (virtual) + 动态绑定，通过 vtable 实现
- 可编写处理未知类型对象的通用代码

5.2 四大设计原则

[p.36]

可复用性 (Reusability)

方法内聚；减少方法规模；保持接口一致性；分离策略与实现；减少耦合；加强封装。

可扩展性 (Extensibility)

封装数据与方法内部结构；用多态替代 switch-case (策略模式)；区分公有/私有方法。

健壮性 (Robustness)

输入校验 (围栏)；先保证正确再优化；检查参数合法性；选择适当实现方法。

协作性 (Collaboration)

详细文档化；类打包成模块；代码易理解。

6 程序规格说明文档及复审

[p.37-38]

程序规格说明 (详细设计说明)：着重描述模块细节——算法过程、数据结构、接口细节、异常处理。简单系统可并入概要设计说明。[p.37]

复审要点 [p.38]：

1. 功能和性能要求是否满足？
2. 描述是否简单清晰？
3. 算法和数据结构是否合理且符合语言特点？
4. 接口是否定义明确一致？
5. 异常和边界条件是否考虑？
6. 是否便于测试和维护？

7 小结

[p.39]

1. **工具选择**：程序流程图（直观灵活）、盒图（强制结构化）、PAD 图（树形 + 自动代码生成）、协作图（对象交互），可组合使用
2. **方法掌握**：JSP 和 LCP 均基于”数据结构驱动设计”，适用于数据处理系统
3. **结构化与 OOP 共性**：都需确定算法和数据结构；OOP 在此之上强调三大特性和四大原则
4. **文档复审**：详细设计说明书是编码直接依据，需通过正式复审

来源：软件工程基础第 7 章详细设计（中南大学任胜兵）

39 页课件全部覆盖 • 2026 年 6 月 1 日